BENNETT UNIVERSITY

School of Computer Science Engineering and Technology

Computational Thinking and Programming

Course Code: 2026CSAE100

PYTHON PROJECT REPORT

Smart Student Performance Analyzer

Student Name	Nikhil
Enrollment Number	S26CSEU0956
Batch / Section	Batch 27 / Section 14
Team Member 2	Vivek
Team Member 3	Pulkit Mittal
Team Member 4	Parth Thakur
Team Member 5	Aarav Mandhan

Academic Year: 2026–27

1 Problem Statement

What is the problem?
Students often have different academic factors such as marks, attendance, and daily study hours, but they may not have a simple way to analyze all these factors together. It can be difficult to identify weak subjects, low attendance, and insufficient study time by looking at the data separately.

Who faces this problem?
This problem is mainly faced by **college and school students** who want to monitor and understand their academic performance.

Why is it useful to solve this problem?
A Smart Student Performance Analyzer can combine marks, attendance, and study hours to generate a clear performance report. It can identify strong and weak subjects, highlight attendance or study-time issues, and provide simple recommendations to help students improve their academic performance.

2 Project Objectives

1. Collect marks, attendance, and daily study-hour data for each subject.
2. Calculate average marks and identify strong and weak subjects.
3. Detect attendance and study-hour issues using simple rules.
4. Generate personalized performance insights and recommendations.
5. Present the analysis in a clear student performance report.

3 Main Features / Modules

The project is divided into input, analysis, recommendation, and report-generation modules.

S. No.	Module / Feature	Description
1	Student Data Input	Collects student name, subjects, marks, attendance, and daily study hours.
2	Performance Analysis	Calculates average marks and identifies highest and lowest performing subjects.
3	Attendance & Study Analysis	Checks attendance and study hours and flags areas needing attention.
4	Recommendation Engine	Generates simple rule-based recommendations for improvement.
5	Performance Report	Displays the analyzed data, performance category, and recommendations.

4 Algorithm / Flowchart / Program Design

Flowchart / Design

Start → Enter student details → Enter subject data → Calculate average marks → Find highest/lowest subject → Check attendance → Check study hours → Generate recommendations → Display performance report → Stop

5 Python Concepts Used

The project uses basic Python programming concepts including variables, data types, conditional statements, loops, lists, dictionaries, functions, file handling, and exception handling where required.

Python Concept	Where / How It Is Used
Variables and Data Types	Store student details, marks, attendance, study hours, and calculated results.
Conditional Statements	Classify performance and generate warnings/recommendations based on rules.
Loops	Process multiple subjects and analyze each subject.
Lists / Tuples / Sets / Dictionaries	Store subject records and organize marks, attendance, and study-hour data.
Functions	Separate input, calculation, analysis, and report-generation tasks.
File Handling	Can be used to save and retrieve student performance records.
Exception Handling	Handles invalid or unexpected input such as non-numeric values.
Libraries Used, if any	Python standard libraries; Flask can be used for the website version.

6 Computational Feature

Describe the main computational or problem-solving feature of your project.

Examples may include:

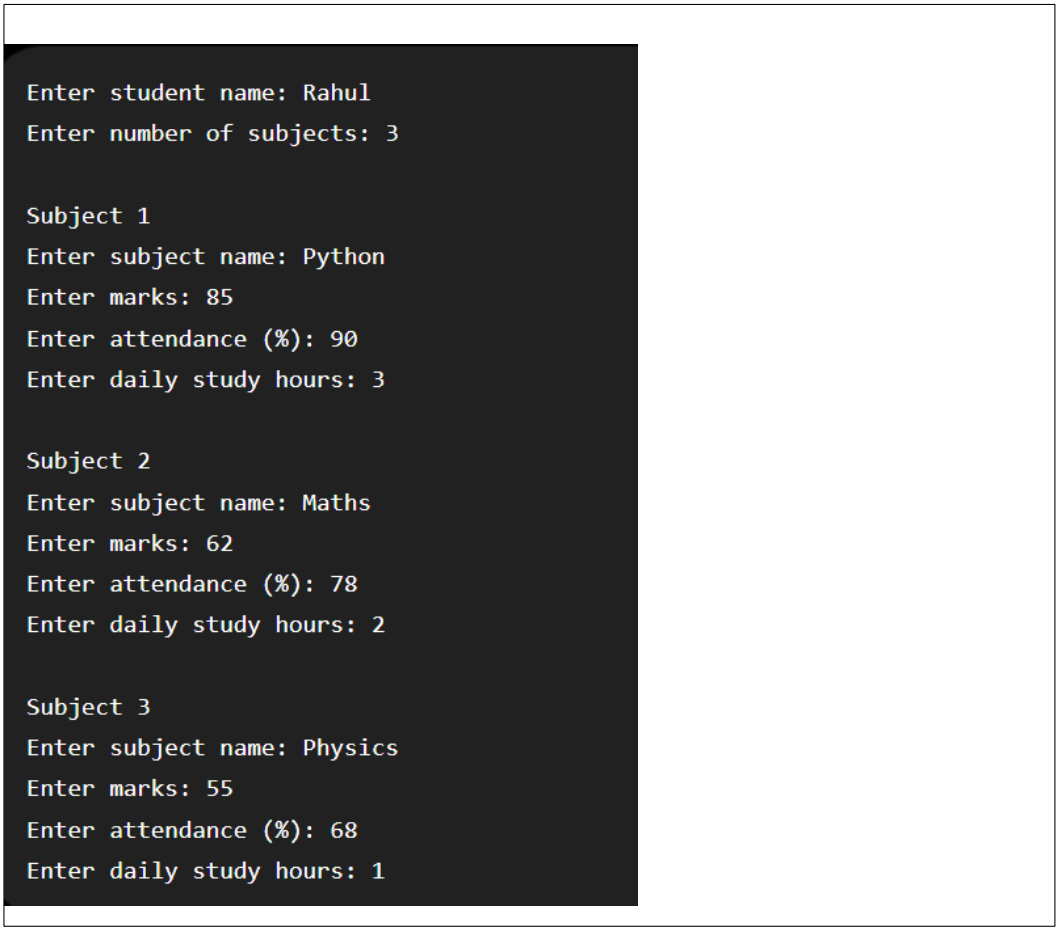
- ranking or scoring;
- analysis or summarization;
- recommendation using simple rules;
- scheduling or allocation;
- conflict detection;
- pattern identification;
- report generation; or
- simple visualization.

The program calculates the average marks, compares subject marks to identify strong and weak subjects, checks whether attendance is below the required threshold, and checks whether study hours are sufficient. Based on these conditions, it generates simple recommendations such as improving attendance, increasing study time, or focusing on a weak subject.

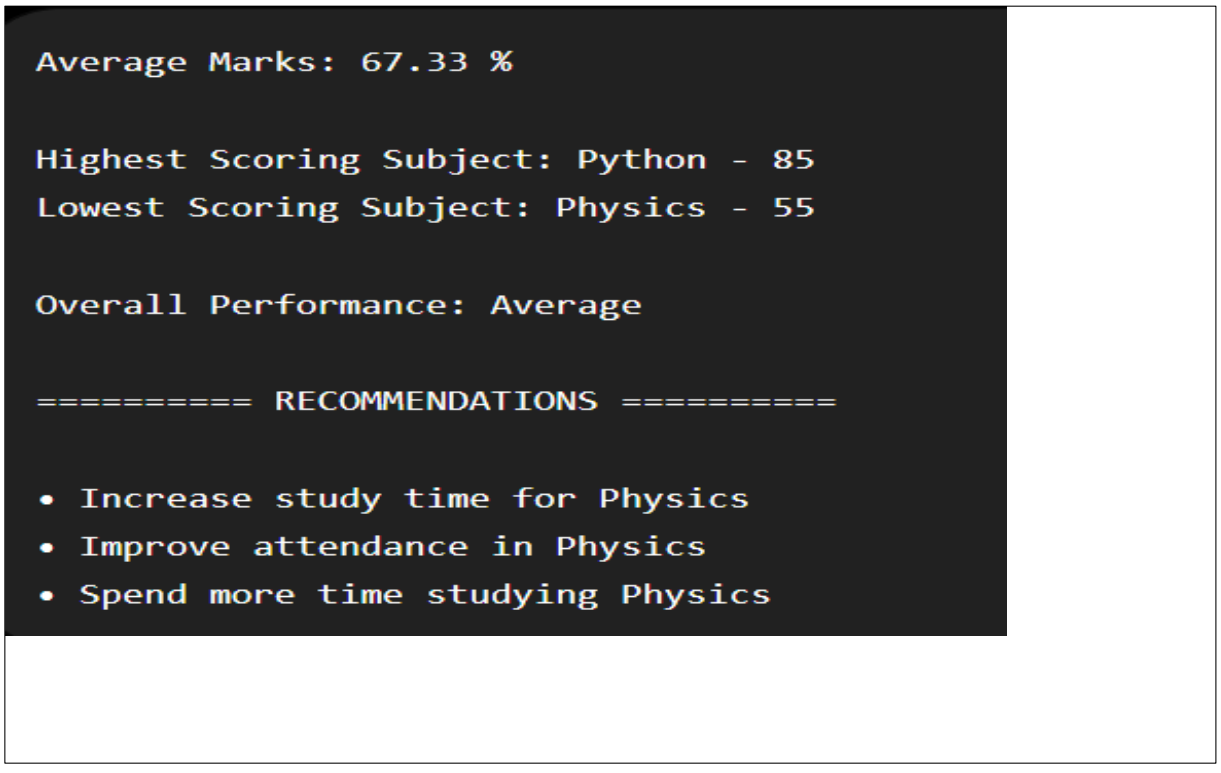
7 Important Outputs / Screenshots

Insert screenshots showing the important parts of the working project.

Screenshot 1



Screenshot 2



8 Testing

Test your project using different types of inputs.

Test	Input	Expected Output	Actual Output	Result
1	Marks = 85%, Attendance = 90%, Study Hours = 3/day	Excellent performance; no warning.	Excellent performance displayed; no warning generated.	Pass
2	Marks = 55%, Attendance = 68%, Study Hours = 1/day	Needs Improvement; attendance and study warnings.	All relevant warnings and recommendations displayed.	Pass
3	Marks = 70%, Attendance = 75%, Study Hours = 2/day	Boundary values accepted; Good performance.	Boundary values processed correctly.	Pass
4	Marks = abc (invalid text)	Invalid input handled without program crash.	Validation/error handling triggered.	Pass
5	Marks = 0%, Attendance = 0%, Study Hours = 0/day	Needs Improvement with relevant recommendations.	Low performance and recommendations displayed.	Pass

Include at least:

- one normal test case;
- one boundary or special test case; and
- one invalid input test case.

9 Team Member Contributions

Clearly mention the contribution of each team member.

Student Name	Enrollment No.	Contribution	Batch / Sectio
Nikhil	S26CSEU0956	Project planning, Python implementation, backend logic, integration, and report preparation.	27 / 14
Vivek	S26CSEU0894	Input/module design, testing, and documentation support.	27 / 14
Pulkit Mittal	S26CSEU0996	Performance-analysis logic, testing, and presentation support.	28 / 14
Parth Thakur	S26CSEU0966	Frontend/web interface support and project documentation.	28 / 14
Aarav Mandhan	S26CSEU0983	Testing, debugging, and presentation support.	28 / 14

10 Challenges Faced and Learning

For your **Smart Student Performance Analyzer**, you can write this section like this:

10. Challenges Faced and Learning

Challenges Faced:

1. One difficulty was handling and organizing multiple inputs such as marks, attendance, and study hours for different subjects.

2. Another challenge was creating proper conditions to analyze the data and generate meaningful recommendations.

How We Solved Them:

- We used **lists and dictionaries** to store subject-wise student data in an organized way.
- We used **conditional statements and loops** to analyze the data, identify weak areas, check attendance and study hours, and generate recommendations.
- We tested the program with different normal, boundary, and invalid inputs to find and fix errors.

New Things We Learned:

- We learned how to apply Python concepts such as **variables, loops, conditional statements, lists, dictionaries, functions, and exception handling** in a real-world project.
- We also learned how to convert raw student data into useful **performance insights and recommendations**.

11 Conclusion and Future Improvements

The Smart Student Performance Analyzer successfully converts student academic inputs into a structured performance report. It helps identify strengths, weaknesses, attendance concerns, study-time issues, and areas requiring improvement.

Future improvements can include a graphical dashboard, database storage for multiple students, subject-wise performance charts, login functionality, and a machine-learning model for performance prediction.

12 References / Acknowledgements

Mention any websites, documentation, datasets, tutorials, libraries, or AI tools consulted during the project.

1.

2. Collect marks, attendance, and daily study hours for each subject.

3. Calculate average marks and identify the highest- and lowest-scoring subjects.

Declaration: We confirm that we understand the submitted project and have acknowledged the external resources used during development.